

The Business Analyst Lab

SQL From Scratch to Professional

1. What is SQL?

SQL (Structured Query Language) is a standard programming language used to store, manage, and retrieve data from relational databases.

It is the most important language in the data world and is used by **Business Analysts, Data Analysts, Data Engineers, Product Managers, and Developers**.

- Structured Query Language
- Used to store, manage, and retrieve data in databases
- Works with relational databases (MySQL, PostgreSQL, SQL Server, Oracle)

2. SQL Basics

→ SQL Syntax

- Statements end with ;
- Case-insensitive (SELECT = select)

→ Data Types

- INT
- VARCHAR
- DATE
- BOOLEAN
- FLOAT / DECIMAL

SQL (Structured Query Language) allows you to interact with databases. Before writing queries, you must understand **syntax rules** and **data types**, because they form the foundation of every SQL command.

SQL DATA TYPES — Detailed Notes

SQL data types define **what kind of data** a column can store. Choosing the right data type improves **storage efficiency**, **performance**, and **data accuracy**.

The Business Analyst Lab

1 INT (Integer)

✓ Meaning

INT is used to store **whole numbers** (positive or negative) without decimal points.

✓ Example values

5, -12, 1000

✓ When to use

- IDs (student_id, order_id)
- Counts (quantity, stock)
- Numeric values that will never need decimals (age, score)

✓ Variants

- TINYINT (smaller range)
- SMALLINT
- BIGINT (very large numbers)

✓ Example

```
age INT,  
quantity INT
```

2 VARCHAR (Variable Character String)

✓ Meaning

VARCHAR stores **text** of variable length. You must specify a maximum size.

Example: VARCHAR(50)

✓ Example values

'John Doe', 'New York', 'Business Analyst'

The Business Analyst Lab

✓ When to use

- Names
- Email addresses
- Locations
- Descriptions
- Any text that varies in length

✓ Why "variable"?

It only stores the **actual** characters entered, not the full allocated space → saves memory.

✓ Example

```
name VARCHAR(100),  
email VARCHAR(50)
```

3 DATE

✓ Meaning

DATE stores **calendar dates** in the format:

YYYY-MM-DD

✓ Example values

```
'2023-05-10'  
'2024-11-29'
```

✓ When to use

- Birthdates
- Order dates
- Joining dates
- Any field storing a specific date

✓ Automatic date functions

- CURRENT_DATE

The Business Analyst Lab

- `NOW()` (Date + Time)

✓ Example

```
order_date DATE,  
dob DATE
```

4 BOOLEAN

✓ Meaning

Stores **True/False** values.

Many SQL systems treat BOOLEAN as:

- TRUE or FALSE
- 1 or 0 internally

✓ Example values

TRUE, FALSE

✓ When to use

- Account active or not
- Payment completed or not
- Is_verified
- Is_deleted
- Flags and binary decisions

✓ Example

```
is_active BOOLEAN,  
payment_done BOOLEAN
```

5 FLOAT / DECIMAL

These are used for **numbers with decimal points**, but they behave differently.

The Business Analyst Lab

FLOAT (Approximate Decimal)

✓ **Meaning**

Stores **approximate** decimal values.
It may **round** the values internally → not 100% accurate.

✓ **Example values**

3.14, 10.456, 2500.75

✓ **When to use**

- Scientific values
- Measurements
- Scores with decimals
- Non-financial numeric data

⚠ **Why be careful?**

FLOAT can store values like 9.99999997 instead of exact numbers.

DECIMAL (Exact Decimal)

✓ **Meaning**

Stores decimal values **exactly** as entered.
Very accurate → perfect for financial calculations.

✓ **Format**

`DECIMAL(precision, scale)`

- **precision** → total digits
- **scale** → digits after decimal point

The Business Analyst Lab

✓ Example

DECIMAL(10,2) means:

- 10 total digits
- 2 digits after decimal
- Example values: 12345678.99, 45.50

✓ When to use

- Money
- Prices
- Tax
- Currency
- Values requiring accuracy

✓ Example

```
price DECIMAL(10,2),
discount DECIMAL(5,2)
```

✦ Summary Table

Data Type	Used For	Example	Notes
INT	Whole numbers	10	No decimals
VARCHAR	Text strings	'Mihir'	Specify length
DATE	Dates	'2024-11-29'	Use YYYY-MM-DD
BOOLEAN	True/False	TRUE	Stored as 1/0 internally
FLOAT	Approx decimal	3.14	Not accurate, avoid for money
DECIMAL	Exact decimal	99.99	Best for finance

◆ 1. SQL Syntax (In Detail)

SQL has simple and readable syntax.
Every SQL query follows a structured pattern.

The Business Analyst Lab

✓ SQL Statements End with Semicolon (;)

The semicolon tells the database engine that a command is complete.

Example:

```
SELECT * FROM employees;
```

Even if you write multiple lines, the query ends only when ; is added.

✓ SQL is Case-Insensitive

This means:

- SELECT = select = Select = SeLeCt
- FROM = from
- WHERE = where

All work the same.

Example:

```
SELECT name FROM employees;  
select name from employees;  
SeLeCt name FrOm employees;
```

All three are treated identically.

 **But table names and column names MAY be case-sensitive** depending on the database.

✓ SQL Ignores Extra Spaces

You can format queries for readability:

Bad formatting:

```
SELECT name,age FROM employees;
```

Good formatting:

```
SELECT  
    name,  
    age
```

The Business Analyst Lab

```
FROM  
  employees;
```

Both work the same, but the second one is professional.

2. SQL DATA TYPES

Data types define what kind of data a column can store. Choosing the correct data type improves performance, storage, and data accuracy.

✦ 1. INT (Integer)

Stores **whole numbers**, without decimals.

Examples:

- 10
- 0
- -25

Used for:

- Age
- Quantity
- Number of items
- IDs

Example SQL column:

```
age INT
```

✦ 2. VARCHAR (Variable Character)

Stores **text/string** data.

You must define the maximum length.

Examples:

- 'Mihir'
- 'Business Analyst'
- 'Bengaluru'

The Business Analyst Lab

Usage:

- Names
- Email
- Address
- Comments

Example:

```
name VARCHAR(50)
```

This means the name can be up to **50 characters long**.

3. DATE

Stores **calendar dates** in the format:

```
YYYY-MM-DD
```

Examples:

- 2025-11-15
- 1999-08-22

Useful for:

- Birthdates
- Booking dates
- Order dates

Example:

```
dob DATE
```

4. BOOLEAN

Stores only:

- **TRUE** or **FALSE**
(or 1 / 0 depending on the database engine)

Examples:

The Business Analyst Lab

- `is_active` BOOLEAN
- `is_verified` BOOLEAN

Used for:

- Yes/No
- True/False
- Active/Inactive

5. FLOAT / DECIMAL

Both store **numbers with decimal values**, but the difference is important.

✓ FLOAT (Approximate Values)

Float stores **approximate** decimal numbers.

Not perfectly precise.

Examples:

- 23.45
- 0.567
- 100.99

Used for:

- Temperature
- Scientific values

✓ DECIMAL (Exact Values)

Used when **precision is required** (money, weight, prices).

Examples:

- 999.99
- 123.450

Syntax:

`DECIMAL(10,2)`

The Business Analyst Lab

Meaning:

- 10 total digits
- 2 digits after decimal

Used for:

- Salary
- Amount
- Price

✓ Why Data Types Matter

Choosing the correct data type ensures:

- Faster query performance
- Less storage usage
- Accurate reporting
- No data loss

Example mistake:

Using VARCHAR for storing salary:

```
salary VARCHAR(20)
```

This will store salary as text — impossible to SUM or AVG correctly.

Correct:

```
salary DECIMAL(10,2)
```

🔑 Final Summary

SQL Syntax:

- Statements end with ;
- Case-insensitive
- Formatting doesn't affect execution

The Business Analyst Lab

SQL Data Types:

Data Type	Used For
INT	Whole numbers
VARCHAR	Text / Strings
DATE	Dates
BOOLEAN	True/False values
FLOAT	Approximate decimals
DECIMAL	Precise decimals (money)

3. Database Objects

- **Database** → Collection of tables
- **Tables** → Rows & Columns
- **Views** → Virtual tables
- **Schema** → Structure of database
- **Index** → Improves search speed

SQL DATABASE OBJECTS — Detailed Notes

Database objects are the **components inside a database** used to store, manage, and retrieve data efficiently.

These objects help define the structure and performance of the database.

1 Database

The Business Analyst Lab

✓ Meaning

A **database** is an organized collection of data stored electronically.

✓ Key points

- Contains **multiple tables** and other objects
- Helps store, manage, and retrieve data
- Ensures consistency, integrity, and security of data
- Accessible through SQL commands

✓ Example

A *Sales Database* may contain:

- Customers table
- Orders table
- Products table
- Payments table

✓ Why important?

- Centralized data storage
- Supports multiple users
- Prevents data duplication
- Makes data easy to analyze

2 Tables

✓ Meaning

A **table** stores data in a structured format using **rows and columns**.

✓ Structure

- **Columns** → fields (name, age, price)
- **Rows** → individual records (actual data)

The Business Analyst Lab

✓ Example

Table: **Employees**

id	name	job_title	salary
1	John	Analyst	50000
2	Mira	Manager	70000

✓ Key points

- Each table usually stores **one type of entity**
 - Columns have data types
 - Primary Key ensures uniqueness
 - Foreign Key creates relationships between tables
-

3 Views

✓ Meaning

A **view** is a **virtual table** created from a SELECT query.
It **does not store data physically**; it shows data derived from one or more tables.

✓ Example

```
CREATE VIEW active_customers AS
SELECT customer_id, name
FROM customers
WHERE is_active = TRUE;
```

✓ Uses

- Simplifies complex queries
- Hides sensitive data (security)
- Provides a customized view for specific users
- Helps reuse queries easily

✓ Important

If the underlying table changes → the view data also changes.

The Business Analyst Lab

4 Schema

✓ Meaning

A **schema** defines the **structure and organization** of a database.

It is like a **folder inside a database** that contains tables, views, functions, indexes, etc.

✓ What does a schema contain?

- Tables
- Views
- Indexes
- Procedures
- Sequences
- Triggers (in some DBs)

✓ Why use schemas?

- Organizes database objects
- Improves data management
- Helps in user access control
- Avoids naming conflicts

✓ Example

Schemas in PostgreSQL:

- `public`
- `sales`
- `analytics`

5 Index

✓ Meaning

An **index** is a data structure used to **increase the speed of data retrieval** from a table.

The Business Analyst Lab

✓ How it works

Works like the **index of a book**:

Instead of scanning every page, you jump directly to the page where the information exists.

✓ Example

```
CREATE INDEX idx_customer_name  
ON customers (name);
```

✓ Benefits

- Faster SELECT queries
- Improves performance on large tables
- Useful for frequently searched columns

⚠ Drawbacks

- Slows down INSERT, UPDATE, DELETE (because index must update also)
- Takes extra storage space

✓ When to use

- Searching by name, email, date
- When filtering happens frequently on a column
- For large tables (10k+ rows)

✦ Summary Table

Object	Meaning	Purpose
Database	Collection of tables & objects	Store and manage data
Table	Rows & Columns	Store structured records
View	Virtual table	Simplify queries & secure data
Schema	Structure of database	Organize and group objects
Index	Search optimizer	Improve query performance

The Business Analyst Lab

4. SQL Commands (CRUD)

C – Create

```
INSERT INTO table_name (col1, col2) VALUES (val1, val2);
```

R – Read

```
SELECT column1, column2 FROM table_name;
```

U – Update

```
UPDATE table_name SET column1 = value WHERE condition;
```

D – Delete

```
DELETE FROM table_name WHERE condition;
```

SQL Commands (CRUD) — Detailed Notes

CRUD stands for **Create, Read, Update, Delete**, which are the **four basic operations** you perform on data stored in a database table.

Every SQL developer, analyst, or beginner must understand these commands clearly.

1 C – CREATE (Insert Data)

✓ Purpose

INSERT is used to **add new records (rows)** into a table.

✓ Syntax

```
INSERT INTO table_name (column1, column2, column3)  
VALUES (value1, value2, value3);
```

The Business Analyst Lab

✓ Example

Add a new employee into the employees table:

```
INSERT INTO employees (name, job_title, salary)
VALUES ('Mihir', 'Business Analyst', 65000);
```

✓ Important Notes

- You must provide values in the **same order** as columns.
 - You can insert into specific columns only:

```
INSERT INTO employees (name, salary)
```
 - ```
VALUES ('Sarah', 70000);
```
  - All missing columns must allow **NULL** or have **default values**.
- 

## **2** R – READ (Select/Retrieve Data)

### ✓ Purpose

**SELECT** is used to **fetch data** from a table. It is the most widely used SQL command.

### ✓ Syntax

```
SELECT column1, column2
FROM table_name;
```

### ✓ Example

Retrieve names and salaries:

```
SELECT name, salary
FROM employees;
```

### ✓ To select all columns

```
SELECT * FROM employees;
```

### ✓ Add conditions

```
SELECT *
```

# The Business Analyst Lab

```
FROM employees
WHERE salary > 60000;
```

## ✓ Sort results

```
SELECT *
FROM employees
ORDER BY salary DESC;
```

## ✓ Limit rows

```
SELECT * FROM employees LIMIT 5;
```

---

## **3** U – UPDATE (Modify Existing Data)

### ✓ Purpose

UPDATE is used to **modify existing records** in a table.

### ✓ Syntax

```
UPDATE table_name
SET column1 = value1, column2 = value2
WHERE condition;
```

### ✓ Example

Update salary for a specific employee:

```
UPDATE employees
SET salary = 75000
WHERE name = 'Mihir';
```

### ✓ Important Notes

- **Always use WHERE** to avoid updating every row accidentally.
- You can update multiple columns:

```
UPDATE employees
SET job_title = 'Senior Analyst', salary = 85000
WHERE id = 3;
```

# The Business Analyst Lab

- Without WHERE (dangerous!):

```
UPDATE employees
SET salary = 90000;
```

This will update **all rows**.

---

## **4** D – DELETE (Remove Data)

### ✓ Purpose

DELETE is used to **remove rows** from a table.

### ✓ Syntax

```
DELETE FROM table_name
WHERE condition;
```

### ✓ Example

Delete an employee record:

```
DELETE FROM employees
WHERE id = 5;
```

### ✓ Important Notes

- Always use a **WHERE** clause to avoid deleting everything.
  - Delete all rows (use carefully):  
`DELETE FROM employees;`
  - To remove entire table structure, use:  
`DROP TABLE employees;`
- 

## ★ CRUD Summary Table

| Operation  | SQL Command | Purpose      |
|------------|-------------|--------------|
| C – Create | INSERT      | Add new data |

# The Business Analyst Lab

| Operation         | SQL Command         | Purpose              |
|-------------------|---------------------|----------------------|
| <b>R – Read</b>   | <code>SELECT</code> | Retrieve data        |
| <b>U – Update</b> | <code>UPDATE</code> | Modify existing data |
| <b>D – Delete</b> | <code>DELETE</code> | Remove data          |

## 5. SELECT Statement (Most Important)

### → Basic Select

```
SELECT * FROM customers;
SELECT name, age FROM customers;
```

### → Aliases

```
SELECT name AS customer_name FROM customers;
```

## 6. WHERE Clause (Filtering)

Operators:

`=, >, <, !=, >=, <=`

`BETWEEN | IN | LIKE | AND | OR | NOT`

```
SELECT * FROM customers WHERE age > 25;
```

The **SELECT statement** is the most commonly used SQL command.  
It is used to **retrieve data** from a table.

---

## **1** Basic SELECT

The SELECT command tells the database **which columns you want to fetch**.

### ✓ Example 1: Select all columns

```
SELECT * FROM customers;
```

### ✓ Explanation

# The Business Analyst Lab

- `SELECT` → tells SQL you want to retrieve data
- `*` → means **all columns**
- `FROM customers` → the data is coming from the `customers` table

## ✓ When to use

- When exploring a new table
- When you need all information
- For debugging and analysis

## ⚠ Note

Avoid using `*` in production queries because it:

- Takes more resources
  - Fetches unnecessary data
  - Slows down performance
- 

## ✓ Example 2: Select specific columns

```
SELECT name, age FROM customers;
```

## ✓ Explanation

- Instead of selecting everything, you choose certain columns
- Makes your query **cleaner and faster**

## ✓ When to use

- When you need only particular information
  - For reports, dashboards, analysis
- 

## **2** Column Aliases (AS)

Aliases are temporary names given to columns or tables **only for the output**.

# The Business Analyst Lab

## ✓ Syntax

```
SELECT column_name AS alias_name
FROM table_name;
```

## ✓ Why Use Aliases?

- To rename columns in output
  - To make results more readable
  - To rename complex calculated columns
  - To shorten long column names
- 

## ✓ Example 3: Alias for readability

```
SELECT name AS customer_name
FROM customers;
```

## ✓ Explanation

- `name` is the actual column name
  - `customer_name` is how it will appear in the result
  - Does **not** change the table structure — only output name changes
- 

## ✓ Example 4: Aliases without AS (optional)

```
SELECT name customer_name
FROM customers;
```

This works the same as using AS.

---

## ✓ Example 5: Alias for calculated columns

```
SELECT price * quantity AS total_amount
FROM orders;
```

## ✓ Explanation

# The Business Analyst Lab

- Calculates a new value (multiplication)
  - Renames it to `total_amount`
- 

## ✓ Example 6: Table Aliases

Aliases also shorten table names.

```
SELECT c.name, o.order_id
FROM customers AS c
JOIN orders AS o
ON c.customer_id = o.customer_id;
```

## ✓ Why use table aliases?

- Makes query shorter
  - Improves readability
  - Useful when joining multiple tables
  - Mandatory in self-joins
- 

## ✦ Summary Table

| Concept          | Example                                       | Purpose                   |
|------------------|-----------------------------------------------|---------------------------|
| Select all       | <code>SELECT * FROM customers;</code>         | Retrieve all columns      |
| Select specific  | <code>SELECT name, age FROM customers;</code> | Retrieve selected columns |
| Alias a column   | <code>SELECT name AS customer_name;</code>    | Rename for clarity        |
| Alias calculated | <code>SELECT qty*price AS total;</code>       | Label expressions         |
| Table alias      | <code>FROM customers AS c</code>              | Shorter, cleaner joins    |

## 7. SQL Functions



# The Business Analyst Lab

→ **Aggregate**(CALCULATIONS ON A GROUP OF ROWS, WE USE WITH GROUP BY, HAVING, WHERE,

- COUNT()- USED FOR COUNT THE NUMBER OF ROWS.

SYNTAX:-

- SUM() – USED FOR TO ADD THE VALUE OF ROWS
- AVG()- USED FOR RETURNS THE AVERAGE VALUE
- MAX()- USED TO FIND THE MAXIMUM VALUE
- MIN() – USED TO FIND THE MINIMUM VALUE

```
SELECT COUNT(*) FROM orders;
```

→ **String**

- UPPER()- USED TO CONVERT TEXT INTO UPPERCASE
- LOWER()- USED TO CONVERT CAPITAL LETTER INTO SMALL LETTER
- CONCAT()- USED FOR JOINS MULTIPLE STRINGS
- LENGTH()- USED TO COUNT CHARACTERS

→ **Date**(EXTRACT DATE OF YEAR, MONTH, DAYS)

- NOW()- USED TO RETURNS CURRENT DATE AND TIME.
- DATE\_ADD() OR DATE\_SUB- ADD OR SUBTRACT DAYS, MONTH, YEARS

SYNTAX:-

```
SELECT DATE_ADD('2024-11-28', INTERVAL 10 DAY);
```

- DATEDIFF()- FIND DIFFERENCE BETWEEN TWO DATES
- SYNTAX:-  
SELECT DATEDIFF('2025-01-01', '2024-12-25');

## SQL FUNCTIONS — Detailed Notes

SQL functions are **pre-built operations** that perform calculations, modify text, handle dates, or return useful information.

They make SQL **more powerful and easier** to write.

---

# The Business Analyst Lab

## **1** Aggregate Functions

Used to perform **calculations on a group of rows**, not individual rows.  
Commonly used with:

- GROUP BY
- HAVING
- WHERE (in some cases)

Aggregate functions return a **single value** for a set of rows.

---

### ✓ 1. COUNT() — Counts Rows

#### 📖 Meaning

Counts the number of rows in a table or the number of non-NULL values in a column.

#### ✓ Syntax

```
SELECT COUNT(*) FROM orders;
```

#### ✓ Example

Count how many orders are placed:

```
SELECT COUNT(order_id) FROM orders;
```

---

### ✓ 2. SUM() — Adds Values

#### 📖 Meaning

Returns the **total sum** of a numeric column.

#### ✓ Syntax

```
SELECT SUM(price) FROM sales;
```

# The Business Analyst Lab

## ✓ Example

Find total revenue:

```
SELECT SUM(total_amount) AS total_revenue
FROM orders;
```

---

## ✓ 3. AVG() — Average Value

### 📖 Meaning

Returns the **average** of a numeric column.

### ✓ Syntax

```
SELECT AVG(salary) FROM employees;
```

## ✓ Example

Find average age:

```
SELECT AVG(age) FROM customers;
```

---

## ✓ 4. MAX() — Maximum Value

### 📖 Meaning

Returns the **highest** value in a column.

### ✓ Syntax

```
SELECT MAX(score) FROM results;
```

## ✓ Example

Get the highest price product:

```
SELECT MAX(price) FROM products;
```

---

# The Business Analyst Lab

## ✓ 5. MIN() — Minimum Value

### 📖 Meaning

Returns the **lowest** value in a column.

### ✓ Syntax

```
SELECT MIN(price) FROM products;
```

### ✓ Example

Find youngest customer:

```
SELECT MIN(age) FROM customers;
```

---

## 2 String Functions

String functions are used to **manipulate or analyze text**.

---

## ✓ 1. UPPER() — Convert to Uppercase

### 📖 Meaning

Converts any text into **UPPERCASE**.

### ✓ Syntax

```
SELECT UPPER(name) FROM employees;
```

### ✓ Example

"mihir" → "MIHIR"

---

# The Business Analyst Lab

## ✓ 2. LOWER() — Convert to Lowercase

### 📖 Meaning

Converts text into **lowercase**.

### ✓ Syntax

```
SELECT LOWER(city) FROM customers;
```

### ✓ Example

```
"DELHI" → "delhi"
```

---

## ✓ 3. CONCAT() — Join Multiple Strings

### 📖 Meaning

Combines two or more strings.

### ✓ Syntax

```
SELECT CONCAT(first_name, ' ', last_name) AS full_name
FROM customers;
```

### ✓ Example

```
"Mihir" + "Gaurav" → "Mihir Gaurav"
```

---

## ✓ 4. LENGTH() — Count Characters

### 📖 Meaning

Returns the **number of characters** in a string.

### ✓ Syntax

# The Business Analyst Lab

```
SELECT LENGTH(name) FROM employees;
```

## ✓ Example

"Business" → 8

---

## 3 Date Functions

Date functions are used to **extract, modify, compare, or calculate** dates and time.

---

### ✓ 1. NOW() — Current Date & Time

#### 📖 Meaning

Returns the **current system date and time**.

#### ✓ Syntax

```
SELECT NOW();
```

#### ✓ Output Example

```
2025-11-28 09:15:45
```

---

### ✓ 2. DATE\_ADD() / DATE\_SUB()

#### 📖 Meaning

Adds or subtracts a specific time interval (days, months, years).

#### ✓ Syntax

```
SELECT DATE_ADD('2024-11-28', INTERVAL 10 DAY);
SELECT DATE_SUB('2024-11-28', INTERVAL 2 MONTH);
```

# The Business Analyst Lab

## ✓ Examples

Add 10 days:  
`2024-11-28 + 10 days = 2024-12-08`

Subtract 2 months:  
`2024-11-28 - 2 months = 2024-09-28`

---

## ✓ 3. DATEDIFF() — Difference Between Two Dates

### 📖 Meaning

Returns the **number of days between two dates**.

### ✓ Syntax

```
SELECT DATEDIFF('2025-01-01', '2024-12-25');
```

### ✓ Result Explanation

`2025-01-01 - 2024-12-25 = 7 days`

---

## ✦ Quick Summary Table

| Function Type | Examples                     | Purpose             |
|---------------|------------------------------|---------------------|
| Aggregate     | COUNT, SUM, AVG, MAX, MIN    | Calculate on groups |
| String        | UPPER, LOWER, CONCAT, LENGTH | Modify text         |
| Date          | NOW, DATE_ADD, DATEDIFF      | Handle dates & time |

## 8. GROUP BY & HAVING

# The Business Analyst Lab

```
SELECT city, COUNT(*)
FROM customers
GROUP BY city
HAVING COUNT(*) > 10;
```

## GROUP BY & HAVING in SQL (Detailed Explanation)

GROUP BY and HAVING are used **together** to perform calculations **on groups of rows**, not individual rows.

---

### ✓ 1. What is GROUP BY?

GROUP BY groups rows that have the **same value** in one or more columns.

#### 🔗 Purpose of GROUP BY:

- Combine similar rows
  - Perform **aggregations** (COUNT, SUM, AVG, MAX, MIN)
  - Get **summaries** like:
    - total sales per city
    - number of customers per country
    - average salary per department
- 

### ✦ Basic Syntax

```
SELECT column_name, AGGREGATE_FUNCTION(column_name)
FROM table_name
GROUP BY column_name;
```

---

### 🔗 Example: Count customers in each city

```
SELECT city, COUNT(*)
FROM customers
```



# The Business Analyst Lab

GROUP BY city;

## ✓ What this does:

- Groups all customers **city-wise**
- COUNT(\*) counts how many customers are in each city
- Output looks like:

| city      | count |
|-----------|-------|
| Delhi     | 20    |
| Mumbai    | 15    |
| Pune      | 5     |
| Bengaluru | 18    |

---

## ⊘ Why we use HAVING with GROUP BY?

HAVING is used to filter **after grouping**.

### ! Difference:

- **WHERE** → filters rows **before grouping**
  - **HAVING** → filters groups **after grouping**
- 

## When to use HAVING?

Use HAVING when you want to filter based on:

- COUNT
  - SUM
  - MAX
  - MIN
  - AVG
- (Any aggregate function)

# The Business Analyst Lab

---

## ✈ 2. GROUP BY + HAVING Example

🔗 "Show only those cities where customer count is more than 10"

```
SELECT city, COUNT(*)
FROM customers
GROUP BY city
HAVING COUNT(*) > 10;
```

### ✓ Explanation:

1. GROUP BY city  
→ First groups the customers according to city.
2. COUNT(\*)  
→ Calculates number of customers in each city.
3. HAVING COUNT(\*) > 10  
→ Shows **only those cities** where the count is more than 10.

---

## 📋 Output Example

| city   | count |
|--------|-------|
| Delhi  | 20    |
| Mumbai | 15    |

Cities like Pune (5 customers) are **not shown** because they don't meet the condition.

---

## 🔍 WHERE vs HAVING (Important for Interviews)

# The Business Analyst Lab

| Feature                        | WHERE                               | HAVING           |
|--------------------------------|-------------------------------------|------------------|
| Used before/after GROUP BY     | Before                              | After            |
| Filters rows                   | ✓ Yes                               | ✓ Filters groups |
| Works with aggregate functions | ✗ No                                | ✓ Yes            |
| Example                        | WHERE age > 18 HAVING COUNT(*) > 10 |                  |

---

## 🎯 Example: Use WHERE + GROUP BY + HAVING together

🗒️ "Count customers per city but only consider customers age > 25. Show cities where count > 5"

```
SELECT city, COUNT(*)
FROM customers
WHERE age > 25
GROUP BY city
HAVING COUNT(*) > 5;
```

This is the **correct** order:

1. WHERE → filters rows
  2. GROUP BY → groups remaining rows
  3. HAVING → filters groups
- 

## ★ Final Quick Notes

- GROUP BY = summarize data
- HAVING = filter summarized data
- Always use aggregate functions with HAVING
- WHERE cannot use aggregate functions

# The Business Analyst Lab

## 🔗 9. JOINS (Very Important)

### ➔ Types of Joins

- **INNER JOIN** → Matching rows
- **LEFT JOIN** → All from left + matches
- **RIGHT JOIN** → All from right + matches
- **FULL JOIN** → All rows from both sides
- **SELF JOIN** → Join with same table

SQL JOINS are used to combine rows from **two or more tables** based on a related column.

Example tables:

#### customers

| customer_id | name  | city   |
|-------------|-------|--------|
| 1           | Rahul | Delhi  |
| 2           | Neha  | Mumbai |
| 3           | Aman  | Delhi  |

#### orders

| order_id | customer_id | amount |
|----------|-------------|--------|
| 101      | 1           | 500    |
| 102      | 1           | 300    |
| 103      | 3           | 700    |

---

## 🔥 1. INNER JOIN (Most used)

Shows **only matching rows** from both tables.

### ◆ Syntax:

# The Business Analyst Lab

```
SELECT columns
FROM tableA
INNER JOIN tableB
ON tableA.common_column = tableB.common_column;
```

## ◆ Example:

```
SELECT customers.name, orders.amount
FROM customers
INNER JOIN orders
ON customers.customer_id = orders.customer_id;
```

## ✓ Output:

Only customers who have placed an order:

| name | amount |
|------|--------|
|------|--------|

|       |     |
|-------|-----|
| Rahul | 500 |
|-------|-----|

|       |     |
|-------|-----|
| Rahul | 300 |
|-------|-----|

|      |     |
|------|-----|
| Aman | 700 |
|------|-----|

! Non-matching rows (Neha) are **not shown**.

---

## 2. LEFT JOIN (Left Table = Priority)

Shows **all rows from left table**, and matching rows from right table.  
Non-matching rows = NULL values.

## ◆ Syntax:

```
SELECT *
FROM tableA
LEFT JOIN tableB
ON tableA.id = tableB.id;
```

## ◆ Example:

```
SELECT customers.name, orders.amount
```

# The Business Analyst Lab

```
FROM customers
LEFT JOIN orders
ON customers.customer_id = orders.customer_id;
```

## ✓ Output:

| name | amount |
|------|--------|
|------|--------|

|       |     |
|-------|-----|
| Rahul | 500 |
|-------|-----|

|       |     |
|-------|-----|
| Rahul | 300 |
|-------|-----|

|      |     |
|------|-----|
| Aman | 700 |
|------|-----|

|      |      |
|------|------|
| Neha | NULL |
|------|------|

! Neha has no order → but LEFT JOIN still includes her.

---

## 🔥 3. RIGHT JOIN (Right Table = Priority)

Shows **all rows from right table**, and matching rows from left.

### ◆ Syntax:

```
SELECT *
FROM tableA
RIGHT JOIN tableB
ON tableA.id = tableB.id;
```

### ◆ Example:

```
SELECT customers.name, orders.amount
FROM customers
RIGHT JOIN orders
ON customers.customer_id = orders.customer_id;
```

## ✓ Output:

# The Business Analyst Lab

**name amount**

Rahul 500

Rahul 300

Aman 700

If a row exists in orders but not in customers, RIGHT JOIN will show it with customer info as NULL.

---

## 4. FULL OUTER JOIN (All rows from both)

Shows:

- all rows from left
- all rows from right
- matching rows
- non-matching rows (with NULLs)

### ◆ Syntax:

```
SELECT *
FROM tableA
FULL OUTER JOIN tableB
ON tableA.id = tableB.id;
```

### ✓ Example:

```
SELECT customers.name, orders.amount
FROM customers
FULL JOIN orders
ON customers.customer_id = orders.customer_id;
```

### ✓ Output:

# The Business Analyst Lab

**name amount**

Rahul 500

Rahul 300

Aman 700

Neha NULL

NULL 250

! Some SQL systems (like MySQL) don't support FULL JOIN directly.

---

## 5. SELF JOIN (Join with same table)

Used when **rows inside the same table are related**.

### When do we use SELF JOIN?

- employee reporting to manager
- comparing rows in the same table
- finding duplicates
- linking parent-child records

### ◆ Syntax:

```
SELECT a.column, b.column
FROM table a
JOIN table b
ON a.common_field = b.common_field;
```

---

### Example: Employee → Manager

**employees table:**



# The Business Analyst Lab

**emp\_id name manager\_id**

|   |       |      |
|---|-------|------|
| 1 | Rahul | NULL |
| 2 | Neha  | 1    |
| 3 | Aman  | 1    |
| 4 | Sima  | 2    |

## Self Join Query:

```
SELECT e.name AS employee,
 m.name AS manager
FROM employees e
LEFT JOIN employees m
ON e.manager_id = m.emp_id;
```

## ✓ Output:

**employee manager**

|       |       |
|-------|-------|
| Rahul | NULL  |
| Neha  | Rahul |
| Aman  | Rahul |
| Sima  | Neha  |

Self Join helps to find hierarchical or relational values **within the same table**.

---

## ✦ Summary Table (One Look Revision)

| Join Type    | What It Returns           |
|--------------|---------------------------|
| <b>INNER</b> | Only matching rows        |
| <b>LEFT</b>  | All left + matching right |

# The Business Analyst Lab

| Join Type    | What It Returns               |
|--------------|-------------------------------|
| <b>RIGHT</b> | All right + matching left     |
| <b>FULL</b>  | All rows from both tables     |
| <b>SELF</b>  | Same table joined with itself |

- **CROSS JOIN** → Cartesian product

```
SELECT c.name, o.order_id
FROM customers c
INNER JOIN orders o ON c.id = o.customer_id;
```

## CROSS JOIN (Cartesian Product)

A **CROSS JOIN** returns the **Cartesian product** of two tables.

☞ This means:

**Every row of Table A is combined with every row of Table B.**

If:

- Table A has **5 rows**
- Table B has **4 rows**

Then:

✦ **CROSS JOIN result =  $5 \times 4 = 20$  rows**

It does **NOT** need any condition (no ON clause).

---

## ◆ Syntax

```
SELECT *
FROM tableA
CROSS JOIN tableB;
```

---

# The Business Analyst Lab

## Example Tables

### customers

**id name**

1 Rahul

2 Neha

(2 rows)

### orders

**order\_id amount**

101 500

102 300

103 700

(3 rows)

---

## CROSS JOIN Example

```
SELECT c.name, o.order_id
FROM customers c
CROSS JOIN orders o;
```

✓ **Output ( $2 \times 3 = 6$  rows)**

**name order\_id**

Rahul 101

Rahul 102

# The Business Analyst Lab

name order\_id

Rahul 103

Neha 101

Neha 102

Neha 103

- Every customer combined with every order
- No relationship required
- Usually used for:
  - Generating combinations
  - Creating all dates × products
  - Making test data
  - Matrix-type tables

---

## ! Your Query (Incorrect for CROSS JOIN)

You wrote:

```
SELECT c.name, o.order_id
FROM customers c
INNER JOIN orders o ON c.id = o.customer_id;
```

This is **INNER JOIN**, not **CROSS JOIN**, because it uses:

- ✓ a join condition
- ✓ matches related rows

---

## So the correct **CROSS JOIN** version is:

```
SELECT c.name, o.order_id
FROM customers c
CROSS JOIN orders o;
```

# The Business Analyst Lab

---

## ✦ Difference Between CROSS JOIN and INNER JOIN

| Feature                 | CROSS JOIN                                      | INNER JOIN         |
|-------------------------|-------------------------------------------------|--------------------|
| Matches using condition | ✗ No                                            | ✓ Yes              |
| Result size             | Huge ( $A \times B$ )                           | Only matching rows |
| Use cases               | combinations, testing meaningful business joins |                    |

## ✦ 10. Subqueries

- Query inside another query

```
SELECT name
FROM customers
WHERE id IN (SELECT customer_id FROM orders);
```

## SUBQUERIES (Very Important Topic)

A **subquery** is a *query inside another query*.  
It is also called:

- Inner query
- Nested query
- Secondary query

The *main query* depends on the result returned by the *subquery*.

---

## 🔥 Why do we use Subqueries?

# The Business Analyst Lab

- ✓ To filter data based on another table
  - ✓ To perform calculations before the main query
  - ✓ To avoid complex joins
  - ✓ To check existence of records
  - ✓ To compare aggregated results (MAX, MIN, AVG)
- 

## ★ 1. Basic Subquery Example

Your example:

```
SELECT name
FROM customers
WHERE id IN (SELECT customer_id FROM orders);
```

### ✓ Explanation:

1. The **inner query** runs first:

```
SELECT customer_id FROM orders;
```

This returns a list of customer IDs who have placed orders.

2. The **outer query** then selects names of customers whose `id` exists in that list.

### ✓ Result:

You get **only customers who placed orders**.

---

## ★ 2. Types of Subqueries

### ◆ (A) Single Row Subquery

Returns **one value**.

Example:

# The Business Analyst Lab

```
SELECT name, age
FROM customers
WHERE age = (SELECT MAX(age) FROM customers);
```

🔍 Finds the **oldest customer**.

---

## ◆ (B) Multiple Row Subquery

Returns **multiple values** → use IN.

Example:

```
SELECT name
FROM customers
WHERE id IN (SELECT customer_id FROM orders);
```

---

## ◆ (C) Multiple Column Subquery

Returns **multiple columns**.

Example:

```
SELECT *
FROM employees
WHERE (department, salary) IN
 (SELECT department, MAX(salary) FROM employees GROUP BY department);
```

🔍 Gets **highest paid employee in each department**.

---

## ◆ (D) Correlated Subquery

Runs **for every row** of the outer query.

Example:

```
SELECT name
FROM employees e
WHERE salary > (
 SELECT AVG(salary)
```

# The Business Analyst Lab

```
FROM employees
WHERE department = e.department
);
```

☞ Employee salary compared to **avg salary of the same department**.

---

## ★ 3. Subquery in SELECT Clause (Scalar Subquery)

```
SELECT
 name,
 (SELECT COUNT(*) FROM orders WHERE orders.customer_id = customers.id)
 AS total_orders
FROM customers;
```

→ Shows customer name + how many orders they placed.

---

## ★ 4. Subquery in FROM Clause (Inline View)

Used as a **temporary table**.

```
SELECT department, avg_salary
FROM (
 SELECT department, AVG(salary) AS avg_salary
 FROM employees
 GROUP BY department
) AS t;
```

---

## ★ 5. Subquery with EXISTS (Fastest)

```
SELECT name
FROM customers c
WHERE EXISTS (
 SELECT 1
 FROM orders o
```



# The Business Analyst Lab

```
WHERE o.customer_id = c.id
) ;
```

- ✓ Faster than IN
  - ✓ Works even when subquery returns large data
  - ✓ Checks if at least 1 matching row exists
- 

## ✦ Subqueries vs Joins (Important Interview Point)

| Subquery                   | JOIN                             |
|----------------------------|----------------------------------|
| Easier to understand       | Faster for large data            |
| Good for filtering         | Good for retrieving related data |
| Works well with IN, EXISTS | Good for combining tables        |

---

## 🔗 Summary (One-Line Definitions)

| Type                | Meaning                                    |
|---------------------|--------------------------------------------|
| Subquery            | Query inside another query                 |
| IN Subquery         | Multiple row match                         |
| Scalar Subquery     | Returns a single value                     |
| Correlated Subquery | Depends on outer query                     |
| EXISTS Subquery     | True if inner query returns at least 1 row |

## 📦 11. Constraints

# The Business Analyst Lab

- PRIMARY KEY
- FOREIGN KEY
- UNIQUE
- NOT NULL
- CHECK
- DEFAULT

## SQL CONSTRAINTS (Very Important Topic)

**Constraints** are rules applied to columns in a table to **maintain data accuracy and integrity**.

They ensure:

- ✓ No duplicate values
  - ✓ No missing values
  - ✓ Valid data only
  - ✓ Relationship between tables is correct
- 

### ★ 1. PRIMARY KEY

A **primary key** uniquely identifies each row in the table.

#### ✓ Rules:

- Cannot be NULL
- Cannot have duplicate values
- Only one primary key allowed per table
- Can be **single column** or **multiple column (composite key)**

#### ✓ Example:

```
CREATE TABLE customers (
 customer_id INT PRIMARY KEY,
 name VARCHAR(50)
);
```

#### ✓ Composite Primary Key:

# The Business Analyst Lab

```
PRIMARY KEY (order_id, product_id)
```

☞ Ensures each combination is unique.

---

## ★ 2. FOREIGN KEY

A **foreign key** creates a relationship between two tables.  
It refers to the **primary key of another table**.

✓ Used to maintain referential integrity.

**Example:**

```
CREATE TABLE orders (
 order_id INT PRIMARY KEY,
 customer_id INT,
 FOREIGN KEY (customer_id) REFERENCES customers(customer_id)
);
```

✓ Ensures orders can be created **only for existing customers**.

---

## ★ 3. UNIQUE Constraint

Ensures that a column has **no duplicate values**.

✓ **Difference between UNIQUE and PRIMARY KEY:**

**PRIMARY KEY**

**UNIQUE**

No NULL allowed    NULL allowed

Only 1 primary key    Many unique constraints allowed

**Example:**

```
CREATE TABLE users (
 id INT PRIMARY KEY,
```

# The Business Analyst Lab

```
email VARCHAR(100) UNIQUE
);
```

☞ No two users can have the same email.

---

## ★ 4. NOT NULL Constraint

Ensures that a column **must always have a value**.

It prevents inserting NULL values.

### Example:

```
CREATE TABLE products (
 id INT PRIMARY KEY,
 name VARCHAR(50) NOT NULL
);
```

☞ A product must always have a name.

---

## ★ 5. CHECK Constraint

Ensures that the value **meets a specific condition**.

### Examples:

#### Age must be above 18

```
age INT CHECK (age >= 18)
```

#### Price must be greater than zero

```
price DECIMAL CHECK (price > 0)
```

#### Salary within a range

```
CHECK (salary BETWEEN 10000 AND 50000)
```

# The Business Analyst Lab

🔒 Used to enforce business rules.

---

## ★ 6. DEFAULT Constraint

Automatically assigns a value **when no value is provided**.

### Example:

```
status VARCHAR(20) DEFAULT 'Active'
```

If you insert data without the status column, it will store:

Active

### Full Example:

```
CREATE TABLE employees (
 emp_id INT PRIMARY KEY,
 name VARCHAR(50) NOT NULL,
 joining_date DATE DEFAULT CURRENT_DATE,
 status VARCHAR(20) DEFAULT 'Active'
);
```

🔒 Useful for timestamps, statuses, default flags, etc.

---

## 🔑 Summary Table for Revision

| Constraint         | Purpose                                      |
|--------------------|----------------------------------------------|
| <b>PRIMARY KEY</b> | Uniquely identifies each row, no NULL        |
| <b>FOREIGN KEY</b> | Links 2 tables, enforces valid relationships |
| <b>UNIQUE</b>      | No duplicate values allowed                  |
| <b>NOT NULL</b>    | Value is mandatory                           |
| <b>CHECK</b>       | Adds a rule/condition                        |

# The Business Analyst Lab

| Constraint | Purpose              |
|------------|----------------------|
| DEFAULT    | Sets automatic value |

## 12. Table Management

### → CREATE TABLE

```
CREATE TABLE employees (
 id INT PRIMARY KEY,
 name VARCHAR(50),
 salary INT
);
```

→

### ALTER TABLE

- ADD
- DROP
- MODIFY

```
ALTER TABLE employees ADD email VARCHAR(100);
```

## Table Management in SQL

Table management means **creating, modifying, and deleting tables** in a database. It is one of the most important concepts for SQL and Database Design.

---

### ★ 1. CREATE TABLE

Used to create a **new table** with columns and their data types.

# The Business Analyst Lab

## ◆ Syntax:

```
CREATE TABLE table_name (
 column1 datatype,
 column2 datatype,
 column3 datatype
);
```

## ✓ Example:

```
CREATE TABLE employees (
 id INT PRIMARY KEY,
 name VARCHAR(50),
 salary INT
);
```

## ✓ Explanation:

- `id` → integer, primary key
- `name` → text up to 50 characters
- `salary` → integer

☞ A new table **employees** is created.

---

## ★ 2. ALTER TABLE

ALTER TABLE is used to **modify the structure** of an existing table.

You can:

- ✓ ADD a new column
  - ✓ DROP (remove) a column
  - ✓ MODIFY the datatype or size of an existing column
  - ✓ RENAME a column
  - ✓ ADD constraints (PRIMARY KEY, UNIQUE, etc.)
- 

## ★ 2.1 ADD Column

# The Business Analyst Lab

## ◆ Syntax:

```
ALTER TABLE table_name
ADD column_name datatype;
```

## ✓ Example:

```
ALTER TABLE employees
ADD email VARCHAR(100);
```

☞ Adds a new column `email` to the `employees` table.

---

## ★ 2.2 DROP Column

Used to **remove a column** from a table.

## ◆ Syntax:

```
ALTER TABLE table_name
DROP COLUMN column_name;
```

## ✓ Example:

```
ALTER TABLE employees
DROP COLUMN email;
```

☞ Deletes the `email` column permanently.

⚠ Be careful — data inside the column is also deleted.

---

## ★ 2.3 MODIFY Column (Change Type or Size)

Used to **change datatype, increase size, or apply constraints**.

## ◆ Syntax (MySQL):



# The Business Analyst Lab

```
ALTER TABLE table_name
MODIFY column_name new_datatype;
```

## ✓ Example:

```
ALTER TABLE employees
MODIFY salary DECIMAL(10,2);
```

☞ Changes salary from INT to DECIMAL.

## ◆ Example: Increase VARCHAR length

```
ALTER TABLE employees
MODIFY name VARCHAR(100);
```

---

## ★ 2.4 RENAME Column (Optional)

In MySQL:

```
ALTER TABLE employees
CHANGE COLUMN name full_name VARCHAR(100);
```

In PostgreSQL / Oracle:

```
ALTER TABLE employees
RENAME COLUMN name TO full_name;
```

---

## ★ 3. DROP TABLE

Used to delete an entire table permanently.

### ◆ Syntax:

```
DROP TABLE table_name;
```

Example:

```
DROP TABLE employees;
```

# The Business Analyst Lab

⚠ Removes table + all data

⚠ Cannot be undone

⚠ Use carefully!

---

## ★ Summary (Perfect for Revision)

| Command                   | Purpose          | Example                                    |
|---------------------------|------------------|--------------------------------------------|
| <b>CREATE TABLE</b>       | Create new table | <code>CREATE TABLE employees (...);</code> |
| <b>ALTER TABLE ADD</b>    | Add a column     | <code>ADD email VARCHAR(100)</code>        |
| <b>ALTER TABLE DROP</b>   | Remove a column  | <code>DROP COLUMN email</code>             |
| <b>ALTER TABLE MODIFY</b> | Change datatype  | <code>MODIFY salary DECIMAL(10,2)</code>   |
| <b>DROP TABLE</b>         | Delete table     | <code>DROP TABLE employees</code>          |

## 🚀 13. SQL Advanced Concepts

### ➔ Window Functions

- `ROW_NUMBER()`
- `RANK()`
- `DENSE_RANK()`
- `LAG()`
- `LEAD()`

```
SELECT name, salary,
ROW_NUMBER() OVER(ORDER BY salary DESC)
FROM employees;
```

## SQL Advanced Concepts — Window Functions (In Details)

Window Functions allow you to perform **calculations across rows** without grouping the data.

# The Business Analyst Lab

- ✓ Unlike **GROUP BY**, window functions do **not** collapse rows.
- ✓ They add **extra calculated columns** while keeping all rows.

Window functions use:

```
OVER (PARTITION BY ... ORDER BY ...)
```

---

## □ 1. ROW\_NUMBER()

Gives a **unique sequence number** for each row based on the specified ORDER.

### ◆ Syntax:

```
ROW_NUMBER() OVER (ORDER BY column)
```

### ✓ Example:

```
SELECT
 name,
 salary,
 ROW_NUMBER() OVER (ORDER BY salary DESC) AS row_num
FROM employees;
```

### ✓ Output:

| name | salary | row_num |
|------|--------|---------|
|------|--------|---------|

|       |       |   |
|-------|-------|---|
| Rahul | 90000 | 1 |
|-------|-------|---|

|      |       |   |
|------|-------|---|
| Neha | 80000 | 2 |
|------|-------|---|

|       |       |   |
|-------|-------|---|
| Arjun | 75000 | 3 |
|-------|-------|---|

☞ No two rows get the same number.

### ✦ Use Case:

- Pagination
- Top N salary selection
- Remove duplicate rows

# The Business Analyst Lab

---

## □ 2. RANK()

Assigns rank based on ORDER, but **skips numbers if ties occur**.

### ◆ Syntax:

```
RANK() OVER (ORDER BY column)
```

### Example:

```
SELECT
 name, salary,
 RANK() OVER (ORDER BY salary DESC) AS ranking
FROM employees;
```

### ✓ Output:

| name | salary | rank |
|------|--------|------|
|------|--------|------|

|       |       |   |
|-------|-------|---|
| Rahul | 90000 | 1 |
|-------|-------|---|

|      |       |   |
|------|-------|---|
| Neha | 80000 | 2 |
|------|-------|---|

|       |       |   |
|-------|-------|---|
| Arjun | 80000 | 2 |
|-------|-------|---|

|       |       |   |
|-------|-------|---|
| Karan | 75000 | 4 |
|-------|-------|---|

✓ Neha & Arjun have the same salary → same rank

! Next rank skipped (No rank 3)

### ★ Use Case:

- Assign rankings with gaps (competition ranking)

---

## □ 3. DENSE\_RANK()

Similar to RANK(), but **does not skip numbers**.

# The Business Analyst Lab

## ◆ Syntax:

```
DENSE_RANK() OVER (ORDER BY column)
```

## ✓ Example:

```
SELECT
 name, salary,
 DENSE_RANK() OVER (ORDER BY salary DESC) AS dense_rank
FROM employees;
```

## ✓ Output:

| name | salary | dense_rank |
|------|--------|------------|
|------|--------|------------|

|       |       |   |
|-------|-------|---|
| Rahul | 90000 | 1 |
|-------|-------|---|

|      |       |   |
|------|-------|---|
| Neha | 80000 | 2 |
|------|-------|---|

|       |       |   |
|-------|-------|---|
| Arjun | 80000 | 2 |
|-------|-------|---|

|       |       |   |
|-------|-------|---|
| Karan | 75000 | 3 |
|-------|-------|---|

✓ No skipped ranks.

## ✦ Use Case:

- Rankings without gaps
- Salary slabs / grade assignment

---

## □ 4. LAG()

Returns the **previous row's value**.

## ◆ Syntax:

```
LAG(column, offset) OVER (ORDER BY column)
```

## ✓ Example:

# The Business Analyst Lab

```
SELECT
 name,
 salary,
 LAG(salary, 1) OVER (ORDER BY salary) AS previous_salary
FROM employees;
```

## ✓ Output:

|  | name | salary | previous_salary |
|--|------|--------|-----------------|
|--|------|--------|-----------------|

|   |  |       |      |
|---|--|-------|------|
| A |  | 30000 | NULL |
|---|--|-------|------|

|   |  |       |       |
|---|--|-------|-------|
| B |  | 40000 | 30000 |
|---|--|-------|-------|

|   |  |       |       |
|---|--|-------|-------|
| C |  | 60000 | 40000 |
|---|--|-------|-------|

## ✦ Use Case:

- Compare current value with previous value
- Stock prices
- Month-over-month sales

---

## □ 5. LEAD()

Returns the **next row's value**.

## ◆ Syntax:

```
LEAD(column, offset) OVER (ORDER BY column)
```

## ✓ Example:

```
SELECT
 name,
 salary,
 LEAD(salary, 1) OVER (ORDER BY salary) AS next_salary
FROM employees;
```

## ✓ Output:

# The Business Analyst Lab

name salary next\_salary

A 30000 40000

B 40000 60000

C 60000 NULL

## ★ Use Case:

- Trend analysis
- Forecasting next value
- Customer journey steps

---

## Important Notes

### ✓ Window functions work only with:

`OVER (ORDER BY ...)` or `OVER (PARTITION BY ...)`

### ✓ **PARTITION BY** divides rows into groups without collapsing data.

Example:

```
RANK() OVER (PARTITION BY department ORDER BY salary DESC)
```

This gives ranks **inside each department separately**.

---

## Summary Table (Best for Notes)

| Function            | Meaning       | Tie Handling  | Use Case             |
|---------------------|---------------|---------------|----------------------|
| <b>ROW_NUMBER()</b> | Unique number | No ties       | Deduplication, Top N |
| <b>RANK()</b>       | Ranking       | Skips numbers | Competition ranking  |

# The Business Analyst Lab

| Function     | Meaning            | Tie Handling | Use Case         |
|--------------|--------------------|--------------|------------------|
| DENSE_RANK() | Ranking            | No skip      | Salary slabs     |
| LAG()        | Previous row value | —            | Trend comparison |
| LEAD()       | Next row value     | —            | Forecasting      |

## → CTE (WITH Clause)

```
WITH temp AS (
 SELECT * FROM employees WHERE salary > 50000
)
SELECT * FROM temp;
```

## CTE (Common Table Expression) – WITH Clause (Detailed Notes)

A CTE (Common Table Expression) is a **temporary result set** that you can reference **within the same SQL query**.

It is created using the **WITH** keyword.

A CTE makes complex queries **cleaner, readable, structured**, and easier to debug.

---



## Why Use CTE?

CTEs are useful when:

- ✓ The query is long and complex
  - ✓ You want to break a query into smaller parts
  - ✓ You want to reuse a temporary result set
  - ✓ You want to perform recursive queries
  - ✓ Replaces subqueries to make the code cleaner
-



# The Business Analyst Lab



## CTE Syntax

```
WITH cte_name AS (
 SELECT columns
 FROM table
 WHERE condition
)
SELECT *
FROM cte_name;
```

- cte\_name = temporary table name
  - It exists **only during the query execution**
  - You can use multiple CTEs separated by commas
- 



## Example

```
WITH temp AS (
 SELECT *
 FROM employees
 WHERE salary > 50000
)
SELECT *
FROM temp;
```

### Explanation:

1. A CTE named **temp** is created.
  2. It stores all employees with salary > 50,000.
  3. Main query selects data from **temp**.
- 



## Multiple CTEs Example

```
WITH high_salary AS (
 SELECT id, name, salary
 FROM employees
 WHERE salary > 60000
) ,
avg_salary AS (
 SELECT AVG(salary) AS avg_sal
```

# The Business Analyst Lab

```
 FROM employees
)
 SELECT h.name, h.salary, a.avg_sal
 FROM high_salary h, avg_salary a;
```

---

## Recursive CTE

Used for:

- ✓ Hierarchical data (manager → employee)
- ✓ Generating series (1,2,3...100)
- ✓ Tree / parent-child relationships

Example:

```
WITH RECURSIVE numbers AS (
 SELECT 1 AS num
 UNION ALL
 SELECT num + 1
 FROM numbers
 WHERE num < 10
)
SELECT * FROM numbers;
```

Creates numbers from 1 to 10.

---

## ★ Advantages of CTE

| Feature                     | Benefit                              |
|-----------------------------|--------------------------------------|
| Readability                 | Cleaner SQL                          |
| Reusability                 | Use CTE multiple times in main query |
| Complex logic               | Break into multiple steps            |
| Recursive queries Supported |                                      |
| Performance                 | Sometimes faster than subqueries     |

---

# The Business Analyst Lab

## Notes Summary (Exam + Interview Friendly)

- CTE is a **temporary named result set** created using `WITH`.
- Helps in simplifying complex SQL queries.
- Exists only during the execution of the query.
- Supports **recursive queries**.
- Improves readability and maintainability compared to subqueries.

### → Stored Procedures

```
CREATE PROCEDURE GetEmployees AS
SELECT * FROM employees;
```

## Stored Procedures – Detailed Notes

A **Stored Procedure** is a **pre-written SQL code** that is stored in the database and can be executed whenever needed.

It is like saving your SQL query as a **function** that you can call again and again.

---

## ★ Why Use Stored Procedures?

Stored Procedures are used because they:

- ✓ Improve performance (pre-compiled and cached)
  - ✓ Reduce code repetition (write once, reuse many times)
  - ✓ Improve security (limit what users can access)
  - ✓ Help in complex business logic
  - ✓ Reduce network traffic (run on the server, not client)
-

# The Business Analyst Lab



## Stored Procedure Syntax

```
CREATE PROCEDURE procedure_name
AS
BEGIN
 SQL statements;
END;
```

To run the procedure:

```
EXEC procedure_name;
```

---



## Example: Simple Stored Procedure

```
CREATE PROCEDURE GetEmployees
AS
BEGIN
 SELECT * FROM employees;
END;
```

### ✓ What this does:

- Creates a stored procedure named **GetEmployees**
- When executed, it returns all rows from the **employees** table

Run it:

```
EXEC GetEmployees;
```

---



## Stored Procedure With Parameters

Stored Procedures can take **inputs (parameters)**.

Example: Get employees with salary greater than X

```
CREATE PROCEDURE GetHighSalaryEmployees
 @minSalary INT
AS
BEGIN
 SELECT *
 FROM employees
```

# The Business Analyst Lab

```
WHERE salary > @minSalary;
END;
```

Run it:

```
EXEC GetHighSalaryEmployees @minSalary = 50000;
```

---

## ✦ Stored Procedure With Multiple Parameters

```
CREATE PROCEDURE GetEmployeesByDept
 @deptName VARCHAR(50),
 @minSalary INT
AS
BEGIN
 SELECT *
 FROM employees
 WHERE department = @deptName
 AND salary > @minSalary;
END;
```

Run:

```
EXEC GetEmployeesByDept @deptName = 'IT', @minSalary = 30000;
```

---

## ✦ Updating Data Using Stored Procedure

```
CREATE PROCEDURE UpdateSalary
 @empId INT,
 @newSalary INT
AS
BEGIN
 UPDATE employees
 SET salary = @newSalary
 WHERE id = @empId;
END;
```

Run:

```
EXEC UpdateSalary @empId = 10, @newSalary = 70000;
```

---

# The Business Analyst Lab

## Deleting Data Using Stored Procedure

```
CREATE PROCEDURE DeleteEmployee
 @empId INT
AS
BEGIN
 DELETE FROM employees
 WHERE id = @empId;
END;
```

Run:

```
EXEC DeleteEmployee @empId = 5;
```

---

## Modifying a Stored Procedure

```
ALTER PROCEDURE GetEmployees
AS
BEGIN
 SELECT name, salary FROM employees;
END;
```

---

## Dropping a Stored Procedure

```
DROP PROCEDURE GetEmployees;
```

---

## Notes Summary (Easy to Revise)

- Stored Procedure = saved SQL code that can be executed anytime.
- Improves performance and security.
- Can accept **parameters**.
- Used for SELECT, INSERT, UPDATE, DELETE operations.
- Execute using EXEC procedure\_name.
- Modify using ALTER, delete using DROP.

# The Business Analyst Lab

## → Triggers

Runs automatically on insert/update/delete.

## SQL Triggers – Detailed Notes

A **Trigger** is a special database object that **automatically executes** (fires) when an event happens on a table.

- ✓ You **do NOT** call a trigger manually
- ✓ It runs automatically when **INSERT**, **UPDATE**, or **DELETE** happens

Triggers help enforce rules, maintain logs, validate data, or automate actions.

---

## ★ Why Do We Use Triggers?

Triggers are useful when you want the database to do something **automatically**, such as:

- Maintaining audit logs (who updated what)
  - Automatically updating timestamps
  - Automatically inserting history records
  - Validating data before insertion
  - Restricting unauthorized changes
  - Maintaining backup tables
- 

## Types of Triggers

### **1** BEFORE Trigger

Runs **before** the statement is executed  
Used for validation

Example: check salary > 0 before inserting

---

# The Business Analyst Lab

## AFTER Trigger

Runs **after** the statement is executed  
Used for logging, backup, audit

---

## INSTEAD OF Trigger

Used mainly in **Views**  
Replaces the actual operation

---

## Events That Can Fire a Trigger

A trigger can run automatically on:

- **INSERT**
  - **UPDATE**
  - **DELETE**
- 

## Basic Trigger Syntax

```
CREATE TRIGGER trigger_name
AFTER INSERT
ON employees
FOR EACH ROW
BEGIN
 SQL statements;
END;
```

---

## Example 1: AFTER INSERT Trigger (Audit Log)

Whenever a new employee is added, store the entry in a log table.



# The Business Analyst Lab

```
CREATE TRIGGER LogNewEmployee
AFTER INSERT ON employees
FOR EACH ROW
BEGIN
 INSERT INTO employee_logs (emp_id, action, action_time)
 VALUES (NEW.id, 'INSERT', NOW());
END;
```

## ✓ What happens?

- When a new row is inserted into **employees**,
  - This trigger runs **automatically** and logs the action.
- 

## Example 2: BEFORE INSERT Trigger (Validation)

Do NOT allow salary less than 10000.

```
CREATE TRIGGER CheckSalary
BEFORE INSERT ON employees
FOR EACH ROW
BEGIN
 IF NEW.salary < 10000 THEN
 SIGNAL SQLSTATE '45000'
 SET MESSAGE_TEXT = 'Salary must be >= 10000';
 END IF;
END;
```

- ✓ Prevents invalid salary values
  - ✓ Shows an error instead of inserting incorrect data
- 

## Example 3: AFTER UPDATE Trigger

Log the old and new salary when an employee salary changes.

```
CREATE TRIGGER SalaryChangeLog
AFTER UPDATE ON employees
FOR EACH ROW
BEGIN
```

# The Business Analyst Lab

```
INSERT INTO salary_logs (emp_id, old_salary, new_salary, changed_on)
VALUES (OLD.id, OLD.salary, NEW.salary, NOW());
END;
```

- ✓ Very useful for audit tracking
  - ✓ Captures both old and new values
- 

## Example 4: AFTER DELETE Trigger

Backup deleted records.

```
CREATE TRIGGER BackupDeletedEmployee
AFTER DELETE ON employees
FOR EACH ROW
BEGIN
 INSERT INTO deleted_employees (id, name, salary, deleted_on)
 VALUES (OLD.id, OLD.name, OLD.salary, NOW());
END;
```

- ✓ Prevents loss of important deleted data
- 

## Modifying a Trigger

SQL does not support direct “ALTER TRIGGER” in some databases  
You must:

1. Drop the trigger
  2. Create it again
- 

## Drop Trigger

```
DROP TRIGGER LogNewEmployee;
```

---

# The Business Analyst Lab

## Quick Revision Notes

- Trigger = Auto-executed code on INSERT/UPDATE/DELETE
- Types: **BEFORE**, **AFTER**, **INSTEAD OF**
- Uses: audit, logs, validation, automation
- Access special variables: **NEW** (new row), **OLD** (previous row)
- Must use **DROP** to modify in many SQL systems

## 14. Indexing

- Improves performance
- Types:
  - Normal Index
  - Unique Index
  - Composite Index

## SQL Indexing – Detailed Notes

An **Index** is a database object that **improves the speed of data retrieval** from a table. Think of it like an **index in a book** – it helps you find data **faster**.

 **Important:** Indexes improve **SELECT performance** but may slow down **INSERT, UPDATE, DELETE**, because the index needs to be maintained.

---

## ★ Why Use Indexes?

- Faster data lookup
  - Efficient sorting (ORDER BY)
  - Efficient filtering (WHERE)
  - Useful for large tables
  - Can enforce uniqueness
-

# The Business Analyst Lab



## Types of Indexes

### 1 Normal / Standard Index

- Speeds up search on a column
- Duplicates are allowed

⇨ *Example:*

```
CREATE INDEX idx_employee_name
ON employees (name);
```

- Creates an index on **name** column of **employees** table
  - Queries like `SELECT * FROM employees WHERE name='Rahul'` run faster
- 

### 2 Unique Index

- Ensures **all values in the column are unique**
- Prevents duplicate entries

⇨ *Example:*

```
CREATE UNIQUE INDEX idx_email_unique
ON employees (email);
```

- Now **no two employees** can have the same email
  - Similar to UNIQUE constraint but explicitly creates an index
- 

### 3 Composite / Multi-Column Index

- Index on **more than one column**
- Useful for queries that filter using multiple columns

⇨ *Example:*

```
CREATE INDEX idx_name_salary
ON employees (name, salary);
```

- Helps queries like:

```
SELECT * FROM employees
```

# The Business Analyst Lab

```
WHERE name='Rahul' AND salary>50000;
```

---

## ◆ Other Notes

- Indexes **do not store data** themselves, they store pointers to data
  - Can be used in `WHERE`, `ORDER BY`, `GROUP BY` clauses
  - Should not index **small tables** or **columns with few distinct values** (like gender)
  - Too many indexes can **slow down insert/update/delete**
- 

## Best Practices

1. Index columns that are **frequently searched** or **joined**
  2. Use **unique index** for primary key or unique constraints
  3. Avoid indexing **columns updated frequently**
  4. Consider **composite index** for multi-column filters
- 

## ✦ Summary Table for Revision

| Index Type           | Purpose                          | Example                                                               |
|----------------------|----------------------------------|-----------------------------------------------------------------------|
| Normal /<br>Standard | Speed up searches                | <code>CREATE INDEX idx_name ON employees(name);</code>                |
| Unique               | Prevent duplicates, speed search | <code>CREATE UNIQUE INDEX idx_email ON employees(email);</code>       |
| Composite            | Multi-column searches            | <code>CREATE INDEX idx_name_salary ON employees(name, salary);</code> |

---

## ⚡ Example Usage in Query

```
SELECT * FROM employees
WHERE name='Neha' AND salary>60000;
```

# The Business Analyst Lab

- If a composite index exists on `(name, salary)`, this query will run much faster.

## 15. Normalization

- 1NF
  - 2NF
  - 3NF
  - BCNF
- Reduces redundancy, improves consistency.

## SQL Normalization – Detailed Notes

**Normalization** is a database design technique used to:

- Reduce **data redundancy**
- Improve **data consistency**
- Make database **efficient and organized**

It divides a large table into smaller **related tables** and establishes relationships between them.

---

## ★ Why Normalize?

1. Avoid duplicate data
  2. Save storage space
  3. Improve data integrity
  4. Make **insert, update, delete operations efficient**
- 



## Normal Forms (NF)

Normalization is performed step by step, known as **normal forms**.

---

# The Business Analyst Lab

## 1 First Normal Form (1NF)

**Rule:**

- Each column must contain **atomic (indivisible) values**
- Each row must be **unique**

◆ **Example:**

**Unnormalized Table:**

| StudentID | Name  | Subjects      |
|-----------|-------|---------------|
| 1         | Rahul | Math, Science |
| 2         | Neha  | English       |

**Normalized to 1NF:**

| StudentID | Name  | Subject |
|-----------|-------|---------|
| 1         | Rahul | Math    |
| 1         | Rahul | Science |
| 2         | Neha  | English |

- ✓ Each cell has **atomic value**
- ✓ No repeating groups

---

## 2 Second Normal Form (2NF)

**Rule:**

- Must be in 1NF
- **No partial dependency** → all non-key columns depend on **whole primary key**

◆ **Example:**

# The Business Analyst Lab

Table with composite key (StudentID + SubjectID):

| StudentID | SubjectID | StudentName | SubjectName | Marks |
|-----------|-----------|-------------|-------------|-------|
|-----------|-----------|-------------|-------------|-------|

**Problem:**

- StudentName depends only on StudentID → partial dependency

**Normalized to 2NF:**

**Students Table**

| StudentID | StudentName |
|-----------|-------------|
|-----------|-------------|

|   |       |
|---|-------|
| 1 | Rahul |
| 2 | Neha  |

**Subjects Table**

| SubjectID | SubjectName |
|-----------|-------------|
|-----------|-------------|

|     |         |
|-----|---------|
| 101 | Math    |
| 102 | Science |
| 103 | English |

**Marks Table**

| StudentID | SubjectID | Marks |
|-----------|-----------|-------|
|-----------|-----------|-------|

|   |     |    |
|---|-----|----|
| 1 | 101 | 95 |
| 1 | 102 | 88 |
| 2 | 103 | 92 |

✓ Eliminates partial dependency

---



# The Business Analyst Lab

## 3 Third Normal Form (3NF)

Rule:

- Must be in 2NF
- **No transitive dependency** → non-key columns should not depend on other non-key columns

◆ Example:

Table:

EmployeeID EmpName DeptName DeptHead

Problem:

- DeptHead depends on DeptName → transitive dependency

Normalized to 3NF:

Employees Table

EmployeeID EmpName DeptID

Department Table

DeptID DeptName DeptHead

✓ Non-key columns depend **only on primary key**

---

## 4 Boyce-Codd Normal Form (BCNF)

Rule:

- Must be in 3NF
- Every determinant must be a **candidate key**
- Fixes anomalies when a table has **overlapping candidate keys**

# The Business Analyst Lab

## ◆ Example:

Table:

Course Instructor Room

Problem:

- Each Room has **one Instructor** → Instructor determines Room
- But primary key is (Course, Room) → violates BCNF

Normalized:

Course Table

Course Instructor

Room Table

Instructor Room

✓ Eliminates anomalies completely

---

## Quick Summary Table

| Normal Form | Requirement                          | Example / Purpose                                |
|-------------|--------------------------------------|--------------------------------------------------|
| 1NF         | Atomic values, unique rows           | Split multiple subjects into separate rows       |
| 2NF         | No partial dependency                | Create separate Students, Subjects, Marks tables |
| 3NF         | No transitive dependency             | Separate Department info from Employee table     |
| BCNF        | Every determinant is a candidate key | Handle overlapping candidate keys                |

---

# The Business Analyst Lab

## Key Points

- Normalization reduces **redundancy**
- Improves **data consistency & integrity**
- Makes **CRUD operations efficient**
- Higher normal forms → more tables, less redundancy

## 16. Real-Time SQL Use Cases

- Reporting (Power BI, Tableau)
- Dashboards
- Data cleaning
- Transformations
- Customer insights
- Sales analysis

## Real-Time SQL Use Cases

SQL is not just a query language; it is the backbone of **data-driven decision-making**. Businesses use SQL to extract, transform, and analyze data to make **real-time decisions**.

Here are the most common use cases:

---

### Reporting (Power BI, Tableau)

#### What it is:

- SQL extracts data from databases and feeds it into **BI tools** like Power BI or Tableau.
- Generates **daily, weekly, or monthly reports** automatically.

#### Example:

```
SELECT department, COUNT(*) AS total_employees
FROM employees
GROUP BY department;
```

# The Business Analyst Lab

- Can be visualized in a **bar chart** in Power BI showing **employee count per department**.

## ◆ Why useful:

- Helps managers track KPIs (Key Performance Indicators)
  - Reduces manual report preparation
  - Enables **real-time insights**
- 

## 2 Dashboards

### ✓ What it is:

- Dynamic dashboards show **real-time data** using SQL queries.
- Includes **sales, revenue, website analytics, customer engagement**.

### ◆ Example:

```
SELECT product_id, SUM(quantity) AS total_sold
FROM orders
WHERE order_date >= CURRENT_DATE - INTERVAL '30 DAY'
GROUP BY product_id;
```

- Power BI/Tableau can show **top-selling products in the last 30 days**.

### ◆ Why useful:

- Helps executives make **quick business decisions**
  - Monitors performance metrics continuously
- 

## 3 Data Cleaning

### ✓ What it is:

- SQL is used to **remove duplicates, correct errors, and format data**.
- Ensures clean, consistent, and usable data for analytics or ETL processes.

### ◆ Example:

# The Business Analyst Lab

```
-- Remove duplicates
DELETE FROM customers
WHERE id NOT IN (
 SELECT MIN(id)
 FROM customers
 GROUP BY email
);
-- Standardize names
UPDATE customers
SET name = UPPER(name);
```

## ◆ Why useful:

- Ensures accurate reporting
  - Prevents errors in analytics
  - Essential for **data pipelines and warehouses**
- 

## **4** Data Transformations

### ✓ What it is:

- SQL can **transform raw data** into structured formats for analysis.
- Includes **aggregations, calculations, joins, and formatting**.

## ◆ Example:

```
SELECT customer_id,
 SUM(quantity * price) AS total_spent,
 COUNT(DISTINCT order_id) AS orders_count
FROM orders
GROUP BY customer_id;
```

- Output can be used to calculate **customer lifetime value (CLV)**.

## ◆ Why useful:

- Prepares **ready-to-analyze datasets**
  - Essential for ETL (Extract, Transform, Load) processes
- 

## **5** Customer Insights

# The Business Analyst Lab

## ✓ What it is:

- SQL helps businesses **understand customer behavior**.
- Example: purchase history, frequency, preferences.

## ◆ Example:

```
SELECT customer_id, COUNT(*) AS total_orders, AVG(total_amount) AS
avg_order_value
FROM orders
GROUP BY customer_id
HAVING COUNT(*) > 5;
```

- Identifies **loyal customers** who order frequently

## ◆ Why useful:

- Helps in **targeted marketing**
  - Drives personalized promotions
  - Improves customer retention
- 

## **6** Sales Analysis

## ✓ What it is:

- SQL analyzes **sales data** to optimize inventory, pricing, and strategy.

## ◆ Example:

```
SELECT product_id, SUM(quantity) AS total_quantity_sold, SUM(quantity *
price) AS total_revenue
FROM orders
WHERE order_date >= DATE_TRUNC('month', CURRENT_DATE)
GROUP BY product_id
ORDER BY total_revenue DESC;
```

- Shows **top-performing products** for the current month

## ◆ Why useful:

- Helps sales teams focus on **high-performing products**

# The Business Analyst Lab

- Supports **forecasting and inventory management**
  - Enables **strategic decision-making**
- 

## Key Takeaways

- SQL is the **backbone of analytics, reporting, and decision-making**
- It supports both **real-time dashboards** and **historical data analysis**
- Critical for **Business Intelligence (BI)**, **ETL processes**, and **data-driven strategies**
- Powers tools like **Power BI, Tableau, Looker, Qlik**

## 17. Practice SQL Topics

- 50+ JOIN queries
- Window function exercises
- Case studies
- E-commerce database project
- Banking database project

## 18. Final Checklist to Become SQL Professional

- ✓ Know DDL, DML, DQL
- ✓ Write complex joins
- ✓ Use window functions
- ✓ Create tables, constraints
- ✓ Write subqueries
- ✓ Optimize queries
- ✓ Understand indexing
- ✓ Work with real datasets
- ✓ Build SQL project portfolio

## Final Checklist to Become an SQL Professional

# The Business Analyst Lab

Becoming an SQL professional is not just about writing queries—it's about **understanding databases, optimizing performance, and applying SQL to solve real business problems.**

Here's a step-by-step checklist with details:

---

## 1 Know DDL, DML, DQL

- **DDL (Data Definition Language):** Commands to **create and manage database structures.**
    - Examples: `CREATE TABLE`, `ALTER TABLE`, `DROP TABLE`
    - Purpose: Structure your database
  - **DML (Data Manipulation Language):** Commands to **insert, update, delete data.**
    - Examples: `INSERT`, `UPDATE`, `DELETE`
    - Purpose: Manage your data
  - **DQL (Data Query Language):** Commands to **retrieve data.**
    - Example: `SELECT`
    - Purpose: Analyze and fetch data efficiently
- 

## 2 Write Complex Joins

- SQL joins help **combine data from multiple tables**
- Types to master:
  - `INNER JOIN` → Only matching rows
  - `LEFT JOIN` → All left + matching right
  - `RIGHT JOIN` → All right + matching left
  - `FULL JOIN` → All rows from both
  - `SELF JOIN` → Join table with itself
  - `CROSS JOIN` → Cartesian product

**Goal:** Be comfortable **writing multi-table queries** to fetch meaningful insights.

---

## 3 Use Window Functions

- Advanced SQL skill for **analytics** without collapsing rows
- Common functions:



# The Business Analyst Lab

- `ROW_NUMBER()`, `RANK()`, `DENSE_RANK()` → Ranking
  - `LAG()`, `LEAD()` → Compare previous/next row
  - Use case: Sales ranking, trends, month-over-month comparisons
- 

## 4 Create Tables & Constraints

- Ability to **design structured tables**
  - Understand constraints for **data integrity**:
    - `PRIMARY KEY` → Unique row identifier
    - `FOREIGN KEY` → Maintain relationships
    - `UNIQUE`, `NOT NULL`, `CHECK`, `DEFAULT`
  - Goal: Build **normalized, scalable databases**
- 

## 5 Write Subqueries & CTEs

- **Subqueries**: Queries inside queries
  - **CTEs (WITH clause)**: Temporary result sets for readability and reuse
  - Use case: Complex filters, recursive hierarchies, intermediate calculations
- 

## 6 Optimize Queries

- Ensure queries run efficiently on **large datasets**
  - Techniques:
    - Use proper **indexes**
    - Avoid unnecessary `SELECT *`
    - Use **joins wisely**
    - Use **EXPLAIN PLAN** to analyze performance
  - Goal: Write **fast and scalable queries**
- 

## 7 Understand Indexing

- Indexes speed up **data retrieval**
- Types:

# The Business Analyst Lab

- Normal / Standard Index
  - Unique Index
  - Composite Index
  - Important for **performance tuning**
- 

## 8 Work with Real Datasets

- Practice with **real-world data**:
    - Sales, e-commerce, HR, or public datasets (Kaggle, government datasets)
  - Simulate **business scenarios**:
    - Customer insights, sales analysis, inventory management
- 

## 9 Build SQL Project Portfolio

- Showcase your skills with **hands-on projects**
- Examples:
  - Sales Dashboard with BI integration
  - Customer Segmentation Analysis
  - Employee Management System with SQL backend
  - ETL pipelines using SQL queries
- Share portfolio on **GitHub** or **personal website**